Adaptive Boss Agent - Utility AI Design

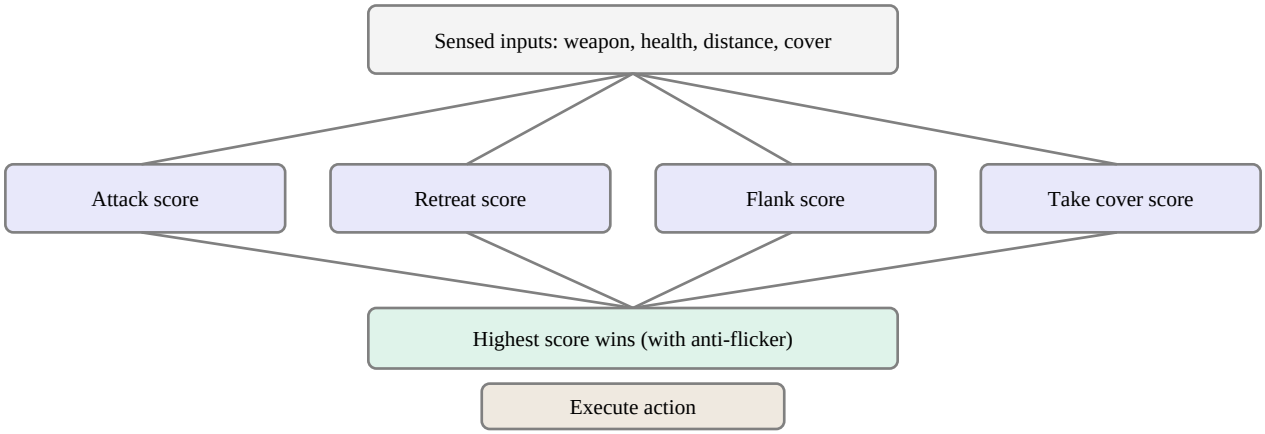
SE3062 Intelligent Systems | Agent owner: Suwasthikka S (IT24101502) | Revised 21 October 2026

1. Overview

The Adaptive Boss uses a Utility AI architecture. Every decision cycle it scores four tactical actions (Attack, Retreat, Flank, Take Cover) from the current game state and performs the highest-scoring one. Unlike a Finite State Machine there are no fixed transitions: the boss re-evaluates every option continuously, so its tactics change as the player's weapon, the player's health, its own health, the distance and cover availability change.

The decision logic is a plain C# class (BossBrain) that knows nothing about GameObjects. A separate MonoBehaviour (BossController) supplies inputs, carries out the chosen action on the NavMesh and records telemetry. This keeps the AI decoupled from visuals. This revision adds the results of play-testing: every run now writes a session summary and a per-decision log (Section 8), and the findings are used to justify the tuning (Section 9).

2. Architecture Diagram



3. Inputs

Input	Range	Notes
Player weapon type	Melee / Ranged	Converted to a ranged-threat value: 0.9 if ranged, 0.1 if melee
Player health	0.0 - 1.0	Fraction of maximum health
Boss health	0.0 - 1.0	Fraction of maximum health
Distance to player	meters	Straight-line distance; 20 m or more counts as far
Cover available	0 / 1	Whether a usable cover point exists (logged as coverAvail). Take Cover is only meaningful when 1.

4. Considerations

A consideration reads one input, applies a response curve and outputs a score between 0 and 1. Each action is a weighted average of its considerations.

Consideration	Formula	Curve / meaning
Closeness	1 - clamp(d / 20)	Linear: 1 when touching, 0 at 20 m or more
Medium band	1 - clamp(|d - 10| / 10)	Triangle peaking at 10 m: the ideal flanking range
Boss hurt	1 - bossHealth	Linear
Retreat urgency	(1 - bossHealth)^2	Quadratic: low until the boss is badly hurt, then rises fast
Player hurt	1 - playerHealth	Linear: a weak player invites aggression
Ranged threat	0.9 ranged / 0.1 melee	Step: the weapon type changes the whole tactic

5. Actions and Weights

Action	Score (weights sum to 1)	Why these weights
Attack	0.4 x bossHealth + 0.4 x closeness + 0.2 x playerHurt	Health and closeness matter equally, since an attack needs strength and reach. Player weakness is a smaller bonus.
Retreat	0.7 x retreatUrgency + 0.3 x closeness	Low health is the main reason to flee. Closeness adds urgency.
Flank	0.5 x rangedThreat + 0.5 x mediumBand	Flanking pays off against ranged weapons, and only at a range where circling is possible.
Take Cover	0.6 x rangedThreat + 0.4 x bossHurt	Cover is mainly a response to ranged weapons. Being hurt makes hiding more attractive.

The base formulas above are confirmed by the logs: in the first decision of the final run (distance 44 m, both at full health, ranged weapon) the logged scores are Attack 0.40, Retreat 0.00, Flank 0.45, Cover 0.54, exactly 0.4, 0, 0.5x0.9, 0.6x0.9. When the boss is at 5% health and far away, Retreat is logged as 0.63 (= 0.7 x 0.95^2) and Cover as up to 0.92 (= 0.6 x 0.9 + 0.4 x 0.95), also matching.

6. Decision Rule (anti-flicker)

Scores are recomputed every frame, so two close scores could make the boss switch many times per second. To prevent this the boss commits to an action:

- It holds the current action for at least 1.0 second (minCommitTime).
- After that it switches only if the best action beats the current one by more than 0.15 (switchMargin).

7. Worked Example

Situation: boss health 30%, player health 80%, ranged weapon, distance 14 m. Closeness = 0.30; medium band = 0.60; boss hurt = 0.70; retreat urgency = 0.49; player hurt = 0.20; ranged threat = 0.9.

Action	Calculation	Score
Attack	0.4(0.30) + 0.4(0.30) + 0.2(0.20)	0.28
Retreat	0.7(0.49) + 0.3(0.30)	0.43
Flank	0.5(0.90) + 0.5(0.60)	0.75
Take Cover	0.6(0.90) + 0.4(0.70)	0.82

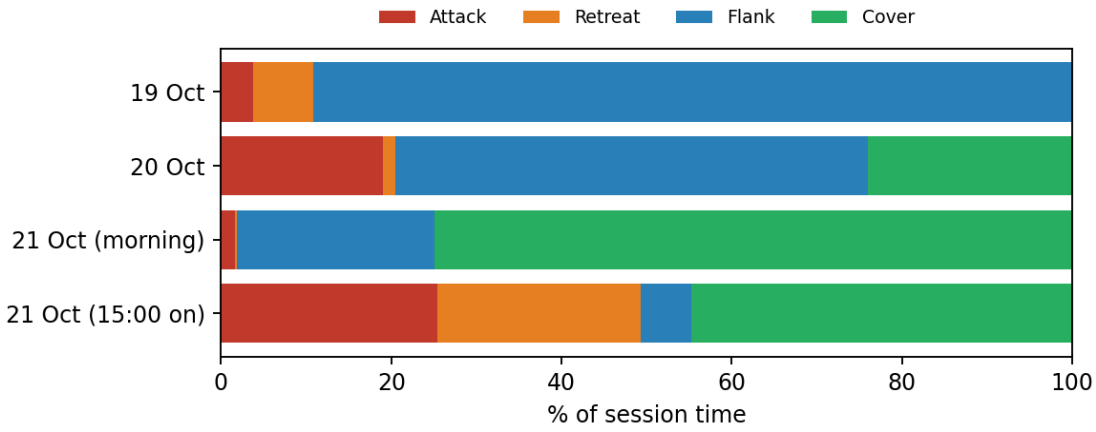
Take Cover scores highest: the player has a ranged weapon and the boss is badly hurt. Anti-flicker: if the boss was already flanking (0.75), Take Cover leads by only 0.07, under the 0.15 margin, so the boss keeps flanking until Take Cover exceeds 0.90.

8. Evaluation from Play-test Telemetry

Every run of the game appends one row to boss_sessions.csv (result, duration, final health, action switches, seconds spent in each action, weapon switches, stuck recoveries). Each run also writes a boss_log_<timestamp>.csv with one row per decision: time, chosen action, weapon, both health values, distance, all four utility scores and cover availability. This makes every decision explainable after the fact.

Data set: 36 sessions logged between 19 and 21 October 2026. 22 ended with the boss defeated, 2 with the player defeated and 12 were quit early. 29 sessions lasted 20 s or more and are used for the action statistics below. Stuck recoveries across all sessions: 0, so the boss never needed the unstuck fallback on the NavMesh.

Test period	Sessions	Switches / min	Attack %	Retreat %	Flank %	Cover %
19 Oct	3	0.9	4	7	89	0
20 Oct	7	1.4	19	1	55	24
21 Oct (morning)	7	1.8	2	0	23	75
21 Oct (15:00 on)	12	8.1	25	24	6	45



9. Findings and Tuning History

The four test periods show how the behaviour changed as the agent was tuned. Period boundaries are taken from the dates in the logs.

- **19 Oct:** the boss spent about 89% of its time flanking and rarely attacked or retreated. Switch counts were near zero, so it behaved like a single-action agent.
- **20 Oct:** cover appeared (24% of time) and the player began switching weapons mid-fight (8 switches logged that day). Flank still dominated and 4 of 7 sessions were quit early.
- **21 Oct morning:** Take Cover dominated at 75% of time. One session lasted 321 s with 302 s in cover and no result; this is a stalemate in which a ranged player and a hiding boss never engage. Sessions were long (mean 141 s) for little gameplay.
- **21 Oct from 15:00:** the stalemate was removed. Switches rose to 8.1 per minute and time was spread over Attack (25%), Retreat (24%) and Cover (45%). 12 sessions: 11 boss defeats and 1 player defeat, with the player finishing at 54% health on average, a fair fight rather than a walkover.

9.1 Stalemate-breaking pressure

The final log shows how the stalemate is avoided. From t = 60 s to 87 s the boss sits at 5% health and the player is at 100%, and the state barely changes (distance 22 to 35 m, Retreat 0.63, Flank 0.19), yet the Attack score climbs from 0.24 to 0.34, 0.69 and 0.80, until the boss attacks at t = 86.6 s. After the player takes damage (t = 98.7 s) Attack drops back to 0.17. So Attack contains an extra term that grows while no damage is exchanged and resets on contact. This is what stops a ranged player and a hiding boss from waiting each other out.

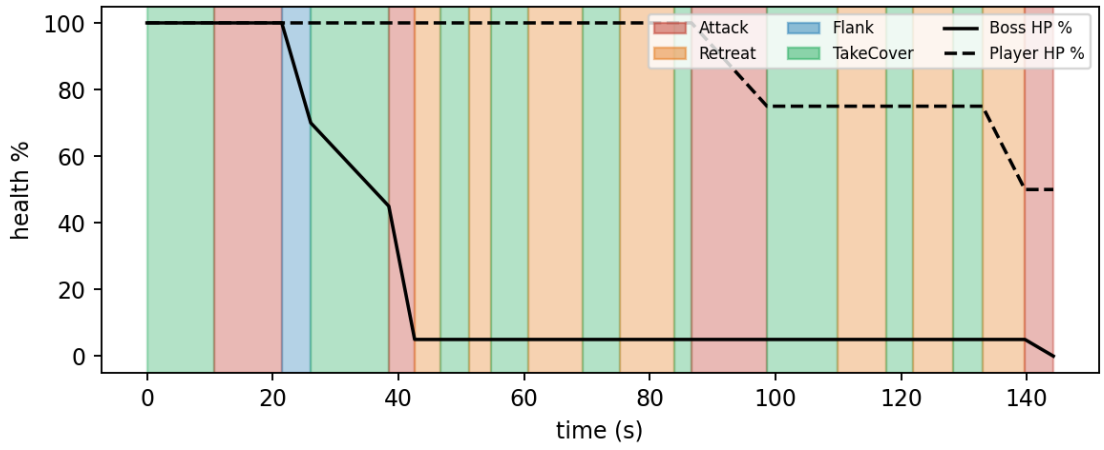


Figure: decision timeline of the final recorded run (boss defeated after 144 s, 20 switches). Background colour is the action being executed.

9.2 Observed limitation

With the boss at 5% health, the log shows Retreat and Take Cover alternating about every 4 to 9 seconds (their scores differ by only 0.1 to 0.3). Both are defensive and the changes respect the 1.0 s commit time and 0.15 margin, so this is not flicker, but it shows that the two actions overlap when the boss is nearly dead. A possible improvement is to merge them or to make Retreat depend on distance from the nearest cover point.

10. Why Utility AI

Alternative	Why Utility AI was preferred
Finite State Machine	Needs a hand-written transition for every pair of states (four actions means up to twelve). Utility AI has none: adding an action means adding one scoring function. The stalemate fix needed one extra term in one score, not new transitions.
Behaviour Tree	Its priority order is fixed, so it cannot easily weigh several factors at once such as weapon type together with health. Utility scores blend them smoothly.
GOAP	Plans multi-step sequences, which is more than this boss needs. It has a higher implementation cost and is harder to debug for four single-step tactics.

Utility AI directly supports the design goal of changing tactics based on the player's weapon and health, and the logged numeric scores make each decision inspectable. This was demonstrated in practice: the stalemate was diagnosed from the logs and fixed without redesigning the agent.

Trade-offs: weights need tuning by play-testing, and scores can be hard to predict when many considerations interact. This is mitigated by keeping considerations few and simple, by the on-screen debug scores, by the CSV telemetry, and by the anti-flicker rule.

11. Mapping to Code

File / output	Responsibility
BossBrain.cs	Pure logic: considerations, action scores, anti-flicker rule. No Unity objects.
BossController.cs	Gathers inputs, calls BossBrain.Evaluate(), performs the movement on the NavMesh, shows the debug overlay and writes telemetry.
boss_sessions.csv	One summary row per play session.
boss_log_<timestamp>.csv	One row per decision, with all four utility scores.